Introduction to Electronic Design Automation

Homework 4

Student: 吳亞倫
ID: B13901011
Department: 電機工程學系

1 Partition

Initial partition

$$A = \{a, b, c\}, \quad B = \{d, e, f\}$$

$$\text{Initial cut cost} = (3 + 3 + 2) + (2 + 4 + 1) + (0 + 2 + 2) = 19$$

Iteration 1

$$A = \{a, b, c\}, \quad B = \{d, e, f\}$$

$$\text{Current cut cost} = 19$$

Step 1

Current I, E, and D values:

Vertex	I	E	$D = E - I$	Locked
a	$2 + 1 = 3$	$3 + 3 + 2 = 8$	$8 - 3 = 5$	no
b	$2 + 0 = 2$	$2 + 4 + 1 = 7$	$7 - 2 = 5$	no
c	$1 + 0 = 1$	$0 + 2 + 2 = 4$	$4 - 1 = 3$	no
d	$1 + 0 = 1$	$3 + 2 + 0 = 5$	$5 - 1 = 4$	no
e	$1 + 1 = 2$	$3 + 4 + 2 = 9$	$9 - 2 = 7$	no
f	$0 + 1 = 1$	$2 + 1 + 2 = 5$	$5 - 1 = 4$	no

Candidate gains:

$$g_{ad} = D_a + D_d - 2c_{ad} = 5 + 4 - 2 \cdot 3 = 3$$

$$g_{ae} = D_a + D_e - 2c_{ae} = 5 + 7 - 2 \cdot 3 = 6$$

$$g_{af} = D_a + D_f - 2c_{af} = 5 + 4 - 2 \cdot 2 = 5$$

$$g_{bd} = D_b + D_d - 2c_{bd} = 5 + 4 - 2 \cdot 2 = 5$$

$$g_{be} = D_b + D_e - 2c_{be} = 5 + 7 - 2 \cdot 4 = 4$$

$$g_{bf} = D_b + D_f - 2c_{bf} = 5 + 4 - 2 \cdot 1 = 7$$

$$g_{cd} = D_c + D_d - 2c_{cd} = 3 + 4 - 2 \cdot 0 = 7$$

$$g_{ce} = D_c + D_e - 2c_{ce} = 3 + 7 - 2 \cdot 2 = 6$$

$$g_{cf} = D_c + D_f - 2c_{cf} = 3 + 4 - 2 \cdot 2 = 3$$

Therefore, the maximum gain is

$$\hat{g}_1 = g_{bf} = 7$$

Temporarily swap b and f, and lock them.

Step 2

Current I, E, and D values:

Vertex	I	E	$D = E - I$	Locked
a	$1+2=3$	$2+3+3=8$	$8-3=5$	no
b	$2+4=6$	$2+0+1=3$	$3-6=-3$	yes
c	$1+2=3$	$0+0+2=2$	$2-3=-1$	no
d	$2+1=3$	$3+0+0=3$	$3-3=0$	no
e	$4+1=5$	$3+2+1=6$	$6-5=1$	no
f	$2+2=4$	$1+0+1=2$	$2-4=-2$	yes

Candidate gains:

$$g_{ad} = D_a + D_d - 2c_{ad} = 5 + 0 - 2 \cdot 3 = -1$$

$$g_{ae} = D_a + D_e - 2c_{ae} = 5 + 1 - 2 \cdot 3 = 0$$

$$g_{cd} = D_c + D_d - 2c_{cd} = -1 + 0 - 2 \cdot 0 = -1$$

$$g_{ce} = D_c + D_e - 2c_{ce} = -1 + 1 - 2 \cdot 2 = -4$$

Therefore, the maximum gain is

$$\hat{g}_2 = g_{ae} = 0$$

Temporarily swap a and e, and lock them.

Step 3

Current I, E, and D values:

Vertex	I	E	$D = E - I$	Locked
a	$2+3=5$	$1+3+2=6$	$6-5=1$	yes
b	$2+2=4$	$0+4+1=5$	$5-4=1$	yes
c	$2+2=4$	$1+0+0=1$	$1-4=-3$	no
d	$3+2=5$	$0+1+0=1$	$1-5=-4$	no
e	$2+1=3$	$3+4+1=8$	$8-3=5$	yes
f	$2+1=3$	$2+1+0=3$	$3-3=0$	yes

Candidate gains:

$$g_{cd} = D_c + D_d - 2c_{cd} = -3 + -4 - 2 \cdot 0 = -7$$

Therefore, the maximum gain is

$$\hat{g}_3 = g_{cd} = -7$$

Temporarily swap c and d, and lock them.

Summary of Iteration 1

$$\hat{g}_1 = g_{bf} = 7,$$
$$\hat{g}_2 = g_{ae} = 0,$$
$$\hat{g}_3 = g_{cd} = -7$$

The partial sums are:

$$G_1 = \sum_{j=1}^{1} \hat{g}_j = 7,$$
$$G_2 = \sum_{j=1}^{2} \hat{g}_j = 7,$$
$$G_3 = \sum_{j=1}^{3} \hat{g}_j = 0$$

Thus,

$$\max_k G_k = 7 \quad \text{at} \quad k = 1$$

Since the largest partial sum is positive, apply the first 1 swap(s):

- Swap b and f.

After applying the selected swap(s),

$$A = \{a, c, f\}, \quad B = \{b, d, e\}$$
$$\text{New cut cost} = 12$$

Iteration 2

Step 1

Current I, E, and D values:

Vertex	I	E	$D = E - I$	Locked
a	$1+2=3$	$2+3+3=8$	$8-3=5$	no
b	$2+4=6$	$2+0+1=3$	$3-6=-3$	no
c	$1+2=3$	$0+0+2=2$	$2-3=-1$	no
d	$2+1=3$	$3+0+0=3$	$3-3=0$	no
e	$4+1=5$	$3+2+1=6$	$6-5=1$	no
f	$2+2=4$	$1+0+1=2$	$2-4=-2$	no

Candidate gains:

$$g_{ab} = D_a + D_b - 2c_{ab} = 5 + -3 - 2 \cdot 2 = -2$$
$$g_{ad} = D_a + D_d - 2c_{ad} = 5 + 0 - 2 \cdot 3 = -1$$
$$g_{ae} = D_a + D_e - 2c_{ae} = 5 + 1 - 2 \cdot 3 = 0$$
$$g_{cb} = D_c + D_b - 2c_{cb} = -1 + -3 - 2 \cdot 0 = -4$$
$$g_{cd} = D_c + D_d - 2c_{cd} = -1 + 0 - 2 \cdot 0 = -1$$

$$g_{ce} = D_c + D_e - 2c_{ce} = -1 + 1 - 2 \cdot 2 = -4$$
$$g_{fb} = D_f + D_b - 2c_{fb} = -2 + -3 - 2 \cdot 1 = -7$$
$$g_{fd} = D_f + D_d - 2c_{fd} = -2 + 0 - 2 \cdot 0 = -2$$
$$g_{fe} = D_f + D_e - 2c_{fe} = -2 + 1 - 2 \cdot 1 = -3$$

Therefore, the maximum gain is

$$\hat{g}_1 = g_{ae} = 0$$

Temporarily swap a and e, and lock them.

Step 2

Current I, E, and D values:

Vertex	I	E	$D = E - I$	Locked
a	$2+3=5$	$1+3+2=6$	$6-5=1$	yes
b	$2+2=4$	$0+4+1=5$	$5-4=1$	no
c	$2+2=4$	$1+0+0=1$	$1-4=-3$	no
d	$3+2=5$	$0+1+0=1$	$1-5=-4$	no
e	$2+1=3$	$3+4+1=8$	$8-3=5$	yes
f	$2+1=3$	$2+1+0=3$	$3-3=0$	no

Candidate gains:

$$g_{cb} = D_c + D_b - 2c_{cb} = -3 + 1 - 2 \cdot 0 = -2$$
$$g_{cd} = D_c + D_d - 2c_{cd} = -3 + -4 - 2 \cdot 0 = -7$$
$$g_{fb} = D_f + D_b - 2c_{fb} = 0 + 1 - 2 \cdot 1 = -1$$
$$g_{fd} = D_f + D_d - 2c_{fd} = 0 + -4 - 2 \cdot 0 = -4$$

Therefore, the maximum gain is

$$\hat{g}_2 = g_{fb} = -1$$

Temporarily swap f and b, and lock them.

Step 3

Current I, E, and D values:

Vertex	I	E	$D = E - I$	Locked
a	$3+2=5$	$2+1+3=6$	$6-5=1$	yes
b	$0+4=4$	$2+2+1=5$	$5-4=1$	yes
c	$0+2=2$	$1+0+2=3$	$3-2=1$	no
d	$3+0=3$	$2+0+1=3$	$3-3=0$	no
e	$4+2=6$	$3+1+1=5$	$5-6=-1$	yes
f	$2+0=2$	$1+2+1=4$	$4-2=2$	yes

Candidate gains:

$$g_{cd} = D_c + D_d - 2c_{cd} = 1 + 0 - 2 \cdot 0 = 1$$

Therefore, the maximum gain is

$$\hat{g}_3 = g_{cd} = 1$$

Temporarily swap c and d, and lock them.

Summary of Iteration 2

$$\hat{g}_1 = g_{ae} = 0,$$
$$\hat{g}_2 = g_{fb} = -1,$$
$$\hat{g}_3 = g_{cd} = 1$$

The partial sums are:

$$G_1 = \sum_{j=1}^{1} \hat{g}_j = 0,$$

$$G_2 = \sum_{j=1}^{2} \hat{g}_j = -1,$$

$$G_3 = \sum_{j=1}^{3} \hat{g}_j = 0$$

Thus,

$$\max_k G_k = 0 \quad \text{at} \quad k = 1$$

Since the largest partial sum is not positive, the algorithm stops.

Final result

$$A = \{a, c, f\}, \quad B = \{b, d, e\}$$

$$\text{Final cut cost} = 12$$

2 Floorplanning

2. (a)

The expression

$$E = 123VH4H56VV78HH.$$

is a valid Polish expression since:

- every operand j, $1 \leq j \leq 8$, appears exactly once in E
- for every prefix of E, $E_i = e_1...e_i, 1 \leq i \leq 2n-1$, the number of operators is less than the number of operands.

	E_1	E_2	E_3	E_4	E_5	E_6	E_7	E_8	E_9	E_{10}	E_{11}	E_{12}	E_{13}	E_{14}	E_{15}
# of Operands	1	2	3	3	3	4	4	5	6	6	6	7	8	8	8
# of Operators	0	0	0	1	2	2	3	3	3	4	5	5	5	6	7

2. (b)

The Polish expression E is not normalized because E contains consecutive operators of the same type: VV and HH.

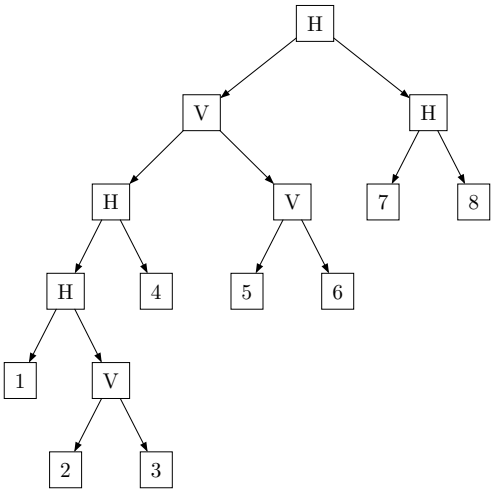


Figure 1: The slicing tree of the given floorplan.

By Figure 1, we can see the slicing tree represented by E. To change the expression into a normalized one, which also means to change the slicing tree into a skewed one, we can apply the following transformation:

- For every node, if its right child is the same type of operator X as itself, left-rotate the subtree rooted at that node.

Figure 2: Left rotation of a subtree with the same type of operator as its parent.

Applying the above transformation to the slicing tree in Figure 1, we can get the following slicing tree:

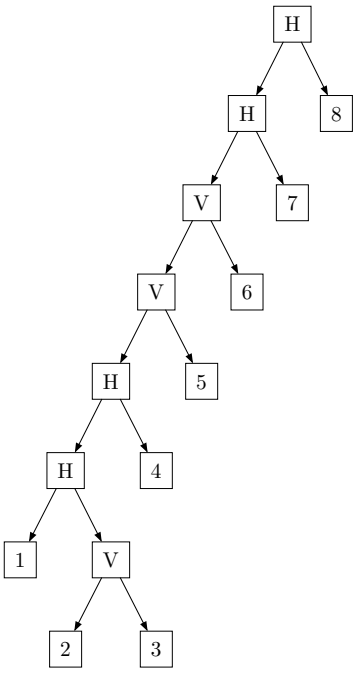


Figure 3: The slicing tree after applying left rotations to all consecutive same-type operator chains.

The normalized Polish expression corresponding to the above slicing tree is

$$E' = 123VH4H5V6V7H8H$$

which is a valid Polish expression and is normalized since it does not contain consecutive operators of the same type. Also, the normalized Polish expression is canonical in representing the floorplan.

2. (c)

For each module, rotation is allowed, so each leaf has two possible shapes. For an internal vertex, the child shape sets are combined by Cartesian product. If the operator is V, then $(w, h) = (w_l + w_r, \max(h_l, h_r))$; if the operator is H, then $(w, h) = (\max(w_l, w_r), h_l + h_r)$. After each combination, dominated shapes are removed. A shape (w, h) is dominated if there exists another shape (w', h') with $w' \leq w$ and $h' \leq h$, and at least one inequality is strict.

For example, for $23V$, we have $S_2 = \{(3, 1), (1, 3)\}$ and $S_3 = \{(4, 3), (3, 4)\}$. Thus,

$$\begin{aligned} S_{23V} &= \text{prune}(\{(3+4, \max(1,3)), (3+3, \max(1,4)), (1+4, \max(3,3)), (1+3, \max(3,4))\}) \\ &= \text{prune}(\{(7,3), (6,4), (5,3), (4,4)\}) \\ &= \{(5,3), (4,4)\}. \end{aligned}$$

Here $(7, 3)$ is dominated by $(5, 3)$, and $(6, 4)$ is dominated by $(4, 4)$.

The full bottom-up computation is:

vertex	candidate shapes before pruning	non-dominated shape set
$23V$	$\{(7,3), (6,4), (5,3), (4,4)\}$	$\{(5,3), (4,4)\}$
$123VH$	$\{(5,5), (7,6), (5,9), (4,10),$ $(5,8), (7,7), (5,12), (4,13)\}$	$\{(5,5), (4,9)\}$
$123VH4H$	$\{(5,7), (4,11), (5,7), (4,11)\}$	$\{(5,7), (4,11)\}$
$56V$	$\{(9,7), (8,7), (12,4), (11,5)\}$	$\{(8,7), (11,5), (12,4)\}$

$123VH4H56VV$	$\{(13,7),(16,7),(17,7),(12,11),$ $(15,11),(16,11)\}$	$\{(13,7),(12,11)\}$
$78H$	$\{(6,7),(4,9),(7,7),(6,6),$ $(3,10),(6,9)\}$	$\{(6,6),(4,9),(3,10)\}$
E	$\{(13,13),(13,16),(13,17),(12,17),$ $(12,20),(12,21)\}$	$\{(13,13),(12,17)\}$

At the root, the two non-dominated shapes have areas $13 \cdot 13 = 169$ and $12 \cdot 17 = 204$. Therefore, the smallest bounding box is 13×13.

Tracing back from the global optimum $(13, 13)$, one compatible set of choices is:

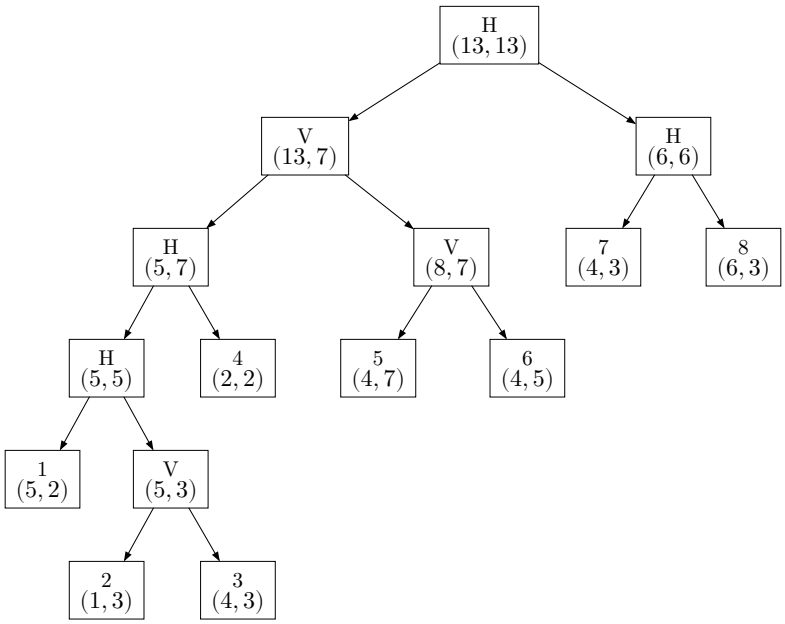


Figure 4: The slicing tree with the chosen shape at each vertex in the final optimum solution.

Hence, one final optimum floorplan has bounding box 13×13.

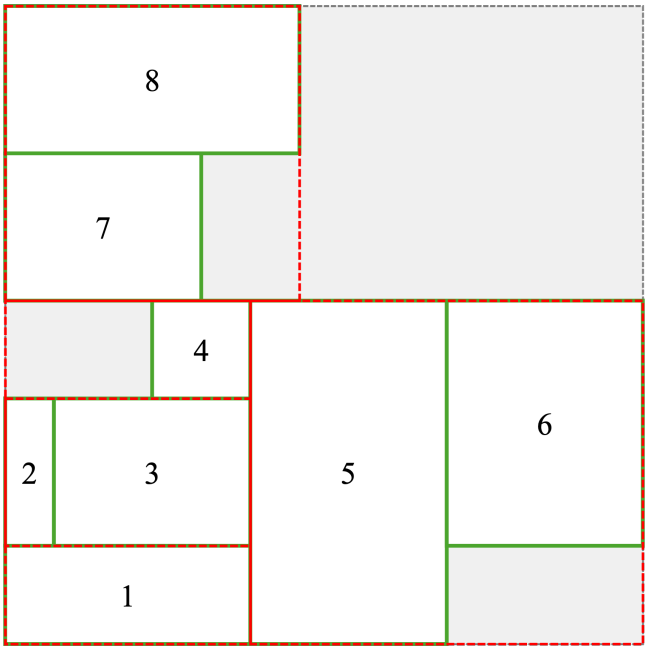


Figure 5: The final optimum floorplan with bounding box 13×13.

2. (d)

No. If two Polish expressions represent the same slicing floorplan, then they have the same set of feasible shapes and hence the same smallest bounding box.

The reason is that different expressions may only differ by equivalent rearrangements, such as swapping the two children of a cut or regrouping consecutive cuts of the same type. These transformations do not change the possible shape set, because the V-cut rule $(w, h) = (w_l + w_r, \max(h_l, h_r))$ and the H-cut rule $(w, h) = (\max(w_l, w_r), h_l + h_r)$ are unaffected by such equivalent reorderings. Therefore, different answers can occur only when the two expressions actually represent different slicing structures, not the same floorplan.

3 Wire-length Estimation

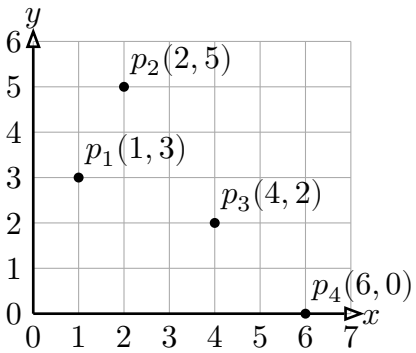


Figure 6: Pins of net N on a grid layout surface.

3. (a) Semi-perimeter Length

The semi-perimeter length is $(6-1)+(5-0)=5+5=10$

3. (b) Complete Graph Length

$$\text{wirelength} \approx \frac{2}{n}\sum_{i,j\in \text{ net}} \text{dist}(i,j) = \frac{2}{4}(3+4+8+5+9+4) = 16.5$$

3. (c) Minimum Spanning Tree Length

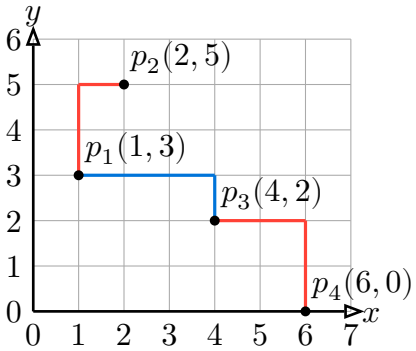


Figure 7: The Minimum Spanning Tree.

The length of the minimum spanning tree is $3+4+4=11$.

3. (d) Minimum Steiner Tree Length

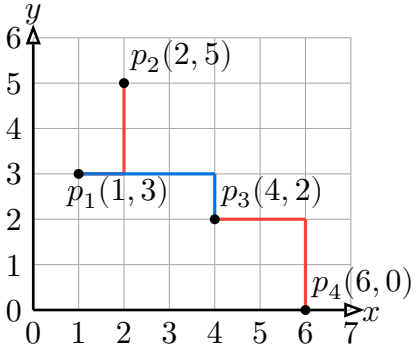


Figure 8: The Minimum Steiner Tree.

The length of the minimum Steiner tree is $2+4+4=10$.

4 Force-Directed Placement

4. (a)

$$\sum_{i\in\{B,C,D,E,F\}} w_{A,i}(x_A - x_i) = 0$$
$$\sum_{i\in\{B,C,D,E,F\}} w_{A,i}(y_A - y_i) = 0$$

$$1(x_A - 0) + 3(x_A - 2) + 1(x_A - 6) + 2(x_A - 8) + 3(x_A - 5) = 0$$
$$1(y_A - 0) + 3(y_A - 5) + 1(y_A - 9) + 2(y_A - 1) + 3(y_A - 4) = 0$$

$$10x_A - 43 = 0 \quad \Rightarrow \quad x_A = 4.3$$
$$10y_A - 38 = 0 \quad \Rightarrow \quad y_A = 3.8$$

Round (x_A, y_A) to the nearest integer grid point, we get the location of A as $(4, 4)$.

4. (b)

The objective function that we want to minimize is the total wire length:

$$\begin{aligned} L &= \sum_{i\in\{B,C,D,E,F\}} w_{A,i}(|x_A - x_i| + |y_A - y_i|) \\ &= 1(|x_A - 0| + |y_A - 0|) + 3(|x_A - 2| + |y_A - 5|) + 1(|x_A - 6| + |y_A - 9|) \\ &+2(|x_A - 8| + |y_A - 1|) + 3(|x_A - 5| + |y_A - 4|) \end{aligned}$$

By using the following python script to enumerate all possible locations of A on the grid that does not overlap with existing points and calculate the corresponding total wire length, we can find the optimal location of A that minimizes the total wire length.

python

```python
w = [1, 3, 1, 2, 3]
x = [0, 2, 6, 8, 5]
y = [0, 5, 9, 1, 4]

cost = 999999999
best_x, best_y = 0, 0

for i in range(10):
    for j in range(10):
        c = 0
        if (i, j) in zip(x, y):
            continue
        for k in range(5):
            c += w[k] * (abs(x[k] - i) + abs(y[k] - j))
        if c < cost:
            cost = c
            best_x, best_y = i, j

print("Best location: ({}, {}) with cost {}".format(best_x, best_y, cost))

```

The result shows that the optimal location of A is $(4, 4)$, with cost at 41, which is the same as the result obtained by the force-directed placement method in part (a).

5 Maze Routing

5. (a)

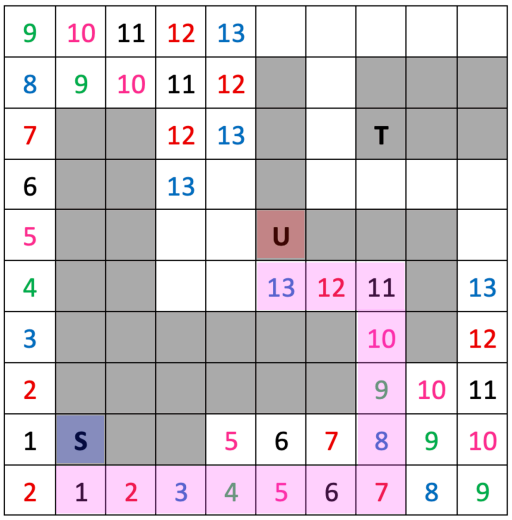
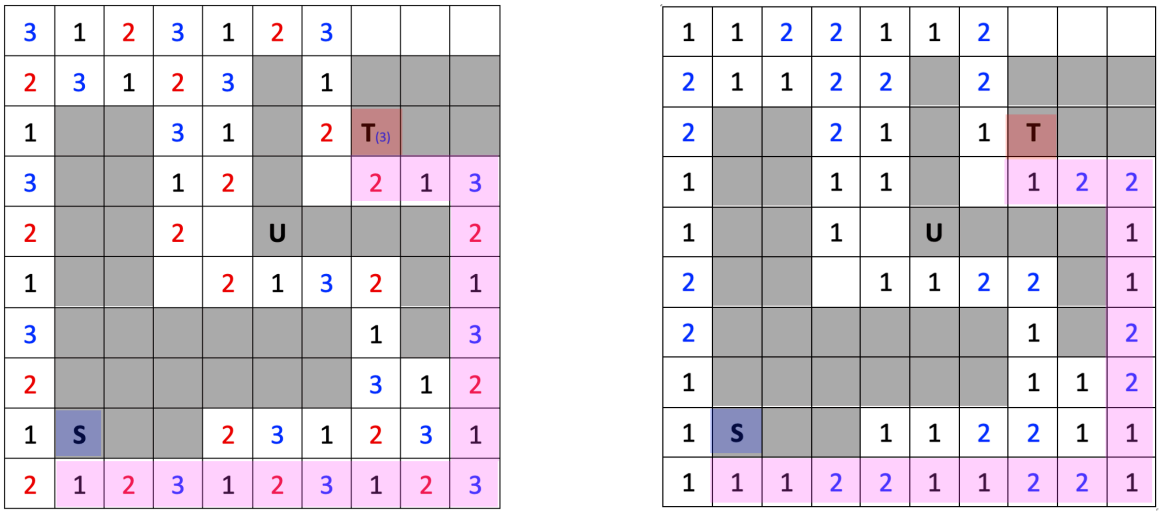


Figure 9: Finding theshortest path between S and U by Lee's approach. The numbers in the grid represent the distance from S. The pink path is the retraced path from U to S.

Lee's propagation labels the reachable grids by their distances from S, as shown in the figure. Retracing from U along adjacent grids with decreasing labels gives the highlighted path.

Hence, the shortest path length from S to U is 14.

5. (b)



(a) Way 1 (b) Way 2

Figure 10: Find a shortest path between S and T by Akers's approach

Figure 10 is the grid after using the Akers's approach to find the shortest path between S and T. The highlighted path in each subfigure is the path found by the algorithm. Both paths are shortest paths with length 18.

In Figure 10a, the grid containing T is labeled 3. Therefore, during retracing, we first move to an adjacent grid labeled 2, then to an adjacent grid labeled 1, then 3, 2, 1, and so on, until reaching S.

On the other hand, in Figure 10b, the grid containing T is labeled as the second 1. Therefore, during retracing, we first move to an adjacent grid labeled 1, then follow the reverse label order $2, 2, 1, 1, 2, 2, ...$ until reaching S.

For both Way 1 and Way 2, the retraced path has length 18. Hence, both paths are shortest paths from S to T.

5. (c)

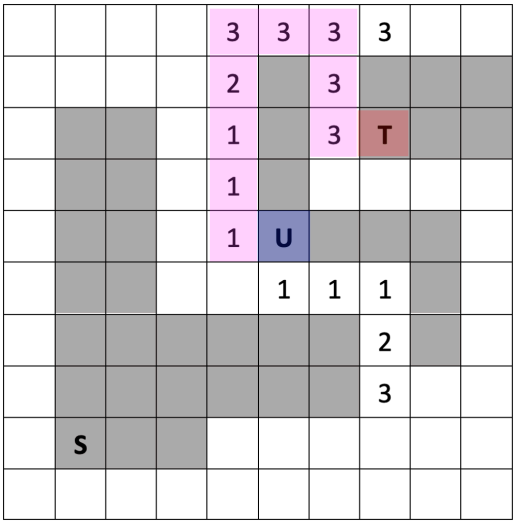


Figure 11: Finding the shortest path between U to T by Hadlock's approach. The number labeled in each grid is the detour number.

Figure 11 shows the result of Hadlock's approach for finding a shortest path from U to T. In the propagation step, U is used as the source. For each expansion, moving to a grid closer to T does not increase the detour number, while moving to a grid farther from T increases the detour number by 1. The grids are expanded in nondecreasing order of detour number until T is reached.

For retracing, we start from T and move backward to U. At each step, we choose an adjacent labeled grid that could be the predecessor under Hadlock's propagation rule. More specifically, if the backward move goes farther from T, the detour number should stay the same; if the backward move goes closer to T, the detour number should decrease by 1.

In Figure 11, the retraced path from T first goes to the adjacent grid labeled 3 on its left. Then it follows the sequence of detour numbers

$$3, 3, 3, 3, 3, 2, 1, 1, 1$$

and finally reaches U.

Equivalently, the path from U to T is: go left once, go up four times, go right twice, go down twice, and go right once. Therefore, the path length is 10.

5. (d)

The minimum distances from S to U is 14, from S to T is 18, and from U to T is 10. Therefore, in the minimum spanning tree, we choose the edges $S - U$ and $U - T$, and do not choose the edge $S - T$. The total length of the minimum spanning tree is $14 + 10 = 24$. The result is shown in Figure 12.

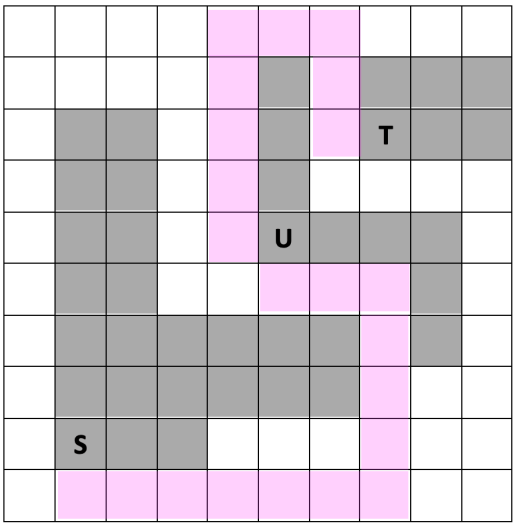


Figure 12: The minimum spanning tree connecting S, U, and T. The total length of the tree is 24.

5. (e)

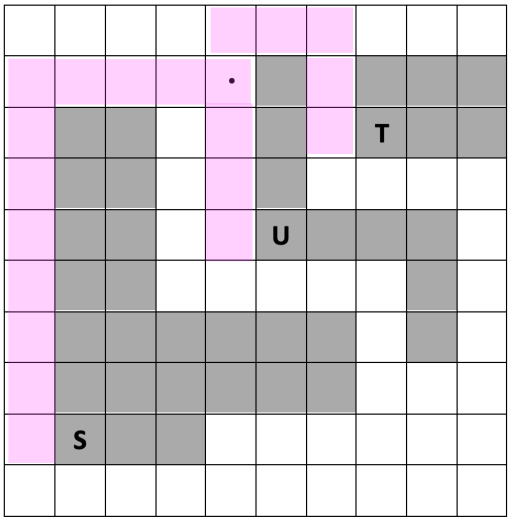


Figure 13: The Steiner tree connecting S, U, and T.

Figure 13 shows a rectilinear Steiner tree connecting S, U, and T. The black dot is chosen as the Steiner point P.

The tree consists of three branches from P to the three terminals. From P to S, the path length is $4 + 7 + 1 = 12$. From P to U, the path length is $3 + 1 = 4$. From P to T, the path length is $1 + 2 + 2 + 1 = 6$.

Therefore, the total net length is

$$12 + 4 + 6 = 22$$

.

6 Channel Routing

6. (a)

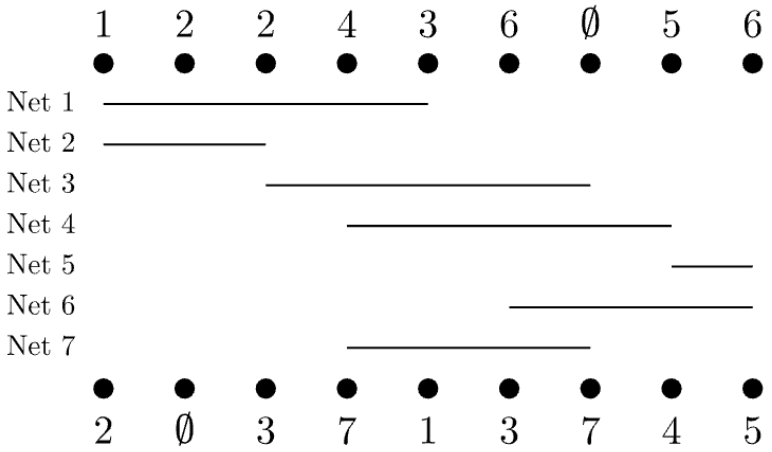
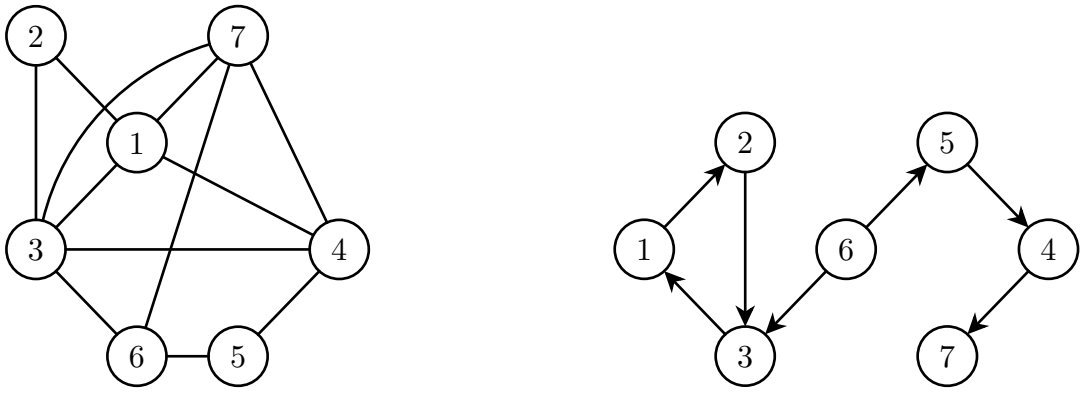


Figure 14: The Horizontal intervals



(a) The Horizontal constraint graph. (b) The Vertical constraint graph.

Figure 15: The constraint graphs for the given channel routing problem.

6. (b)

Basic left-edge algorithm is not applicable, because it ignores vertical constraints.

Constrained left-edge algorithm is also not applicable, because the VCG contains a cycle: $1 \to 2 \to 3 \to 1$.

Therefore, no routing result can be produced by these two algorithms without doglegging.

6. (c)

The cycle in the VCG is $1 \to 2 \to 3 \to 1$. To break the cycle, we can dogleg net 2. Let the new segment of net 2 be $2_a = [1, 2]$ and $2_b = [2, 3]$. After doglegging, the new VCG does not contain any cycle, and the constrained left-edge algorithm can be applied to produce a routing result.

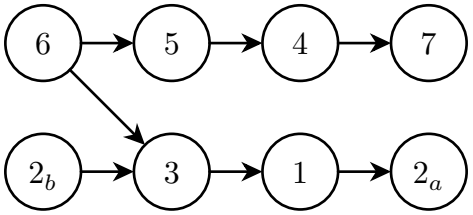


Figure 16: The VCG after doglegging net 2 into 2_a and 2_b.

- Now apply the constrained left-edge algorithm. Let N_x denote net x.
  - Track 1:
    - Initially, the unconstrained intervals are n_{2_b} and n_6.
    - Set `watermark = 0`.
    - Among the unconstrained intervals with left edge greater than `watermark`, choose the one nearest to `watermark`, which is $n_{2_b} = [2, 3]$.
    - Assign n_{2_b} to track 1 and update `watermark = 3`.
    - After removing n_{2_b}, interval $n_6 = [6, 9]$ is still unconstrained and has left edge greater than `watermark`.
    - Assign n_6 to track 1 and update `watermark = 9`.
    - Thus, track 1 contains: n_{2_b}, n_6.
  - Track 2:
    - After removing n_{2_b} and n_6, the unconstrained intervals are n_3 and n_5.
    - Set `watermark = 0`.
    - Choose $n_3 = [3, 6]$.
    - Assign n_3 to track 2 and update `watermark = 6`.
    - Then choose $n_5 = [8, 9]$, since its left edge is greater than `watermark`.
    - Assign n_5 to track 2 and update `watermark = 9`.
    - Thus, track 2 contains: n_3, n_5.
  - Track 3:
    - After removing n_3 and n_5, the unconstrained intervals are n_1 and n_4.
    - Set `watermark = 0`.
    - Choose $n_1 = [1, 5]$.
    - Assign n_1 to track 3 and update `watermark = 5`.
    - Interval $n_4 = [4, 8]$ cannot be assigned to the same track because its left edge is not greater than `watermark`.
    - Thus, track 3 contains: n_1.
  - Track 4:
    - After removing n_1, the unconstrained intervals are n_{2_a} and n_4.
    - Set `watermark = 0`.
    - Choose $n_{2_a} = [1, 2]$.
    - Assign n_{2_a} to track 4 and update `watermark = 2`.
    - Then choose $n_4 = [4, 8]$, since its left edge is greater than `watermark`.
    - Assign n_4 to track 4 and update `watermark = 8`.
    - Thus, track 4 contains: n_{2_a}, n_4.
  - Track 5:
    - After removing n_{2_a} and n_4, the only remaining unconstrained interval is $n_7 = [4, 7]$.
    - Assign n_7 to track 5.
    - Thus, track 5 contains: n_7.
- Therefore, the routing result is:

$$\text{Track } 1 : n_{2_b}, n_6 \quad \text{Track } 2 : n_3, n_5 \quad \text{Track } 3 : n_1$$

$$\text{Track } 4 : n_{2_a}, n_4 \quad \text{Track } 5 : n_7$$

- The resulting channel height is 5 tracks.

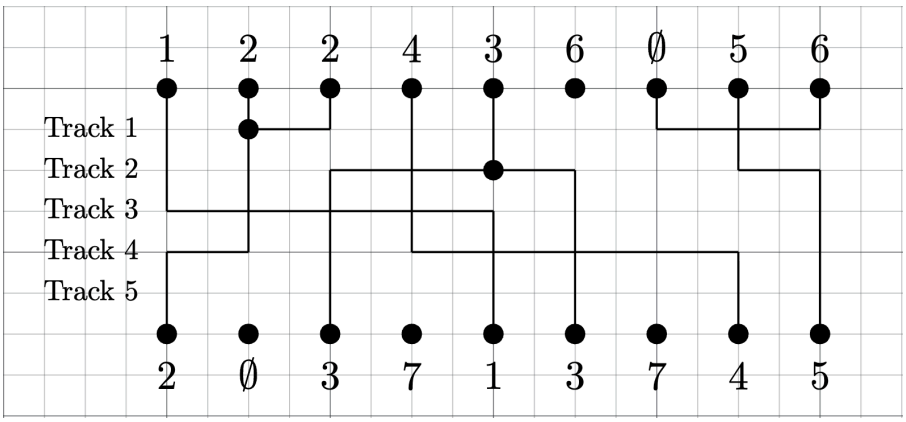


Figure 17: The routing result after doglegging net 2 and applying the constrained left-edge algorithm. The channel height is 5 tracks.

7 Testing

First, the internal signals are

$$y_3 = \overline{x_1 x_2}, \quad y_6 = \overline{x_2 + x_3},$$

and because the fanout branches are functionally identical,

$$y_4 = y_5 = y_3, \quad y_7 = y_8 = y_6.$$

Thus

$$z_1 = z_2 = \overline{y_3 y_6} = x_2 + x_3.$$

After fault equivalence, the relevant fault classes and their detecting test sets are:

Fault class	Detecting tests
y_1 s-a-1	100
y_2 s-a-0	010
x_3 s-a-0	001, 101
x_2 s-a-0	010, 110
x_2 s-a-1	000, 100
y_6 s-a-1	001, 010, 011, 101
z_1 s-a-0	001, 010, 011, 101, 110, 111

By dominance, if the detecting test set of fault f is a subset of that of fault g, then g can be deleted. Therefore,

$$\{100\} \subseteq \{000, 100\}, \quad \{010\} \subseteq \{010, 110\}, \quad \{001, 101\} \subseteq \{001, 010, 011, 101\}.$$

Hence the faults with larger detecting test sets are dominated and can be removed.

Therefore, the prime faults are

$$y_1 \text{ s-a-1}, \quad y_2 \text{ s-a-0}, \quad x_3 \text{ s-a-0}.$$

Their detecting test sets are

$$T(y_1 \text{ s-a-1}) = \{100\}, \quad T(y_2 \text{ s-a-0}) = \{010\}, \quad T(x_3 \text{ s-a-0}) = \{001, 101\}.$$

Thus a complete test set for the prime faults is

$$\{100, 010, 001\}$$

or equivalently

$$\{100, 010, 101\}.$$